SORBONNE UNIVERSITY

Faculté des Sciences et Ingénierie

Parcours Science et Technologie du Logiciel (STL-INSTA)

Analyse d'Algorithmes, Graphes et Applications

Assessment AAGA10 : Composition about the Greedy Connected Dominating Set algorithms

Étudiant:
BOUAFIA Merouane, N°21501802, merouane.bouafia@etu.sorbonne-universite.fr

Mars 2026

Assessment AAGA10: composition about the algorithms presented in [Li, Thai, Wang, Yi, Wan, and Du. Wireless Communications and Mobile Computing, 2005]

1 Introduction

Wireless ad hoc networks consist of mobile nodes that communicate via radio without any fixed infrastructure. A fundamental task in such networks is the construction of a *virtual backbone*: a small subset of nodes through which all other nodes can route their messages. Formally, this corresponds to finding a *Connected Dominating Set* (CDS) of the underlying communication graph.

The communication topology of a wireless network is commonly modeled as a *Unit Disk Graph* (UDG): each node is a point in the plane, and two nodes are neighbors if and only if their Euclidean distance is at most 1 (after normalization of the transmission range). Finding a minimum CDS is NP-hard even on UDGs [2], which motivates the search for efficient approximation algorithms.

Li et al. [1] propose a two-phase greedy algorithm that first computes a dominating set and then connects it using Steiner-tree-like techniques. Their approach achieves a provable approximation ratio and runs in polynomial time. This report is organized as follows. Section 2 formalizes the problem and introduces the relevant data structures. Section 3 presents and analyzes the algorithms of Li et al. Section 4 describes our implementation. Section 5 reports the experimental evaluation. Section 6 gives a critical discussion, and Section 7 concludes.

2 Problem Definition and Data Structures

2.1 Unit Disk Graphs

Definition 1 (Unit Disk Graph)**.** *Given a set $V = \{v_1, \ldots, v_n\}$ of points in $\mathbb{R}^2$, the* Unit Disk Graph *$G = (V, E)$ has an edge $(v_i, v_j) \in E$ if and only if $\|v_i - v_j\|_2 \leq 1$.*

UDGs model idealized wireless networks where every node has the same transmission range. They form a subclass of geometric intersection graphs that admit special structural properties not shared by general graphs.

2.2 Dominating Set and Connected Dominating Set

Definition 2 (Dominating Set)**.** *A subset $D \subseteq V$ is a* Dominating Set *(DS) of $G = (V, E)$ if every vertex in $V \setminus D$ has at least one neighbor in D.*

Definition 3 (Connected Dominating Set)**.** *A dominating set D is a* Connected Dominating Set *(CDS) if the subgraph induced by D in G is connected.*

The Minimum Connected Dominating Set (MCDS) problem asks for a CDS of minimum cardinality. This problem is NP-hard on general graphs [3] and remains NP-hard on UDGs [2].

2.3 Data Structures

The algorithms of Li et al. rely on the following data structures:

- **Adjacency lists** for the UDG G, stored in $O(n + m)$ space where $m = |E|$.
- **Priority queue / max-heap** keyed by vertex degree (or number of uncovered neighbors) for the greedy selection in Phase 1.
- **Union–Find (Disjoint Set Union)** structure for efficiently tracking connected components during Phase 2.
- **BFS/DFS trees** used for computing shortest paths or Steiner nodes when connecting the dominating set.
- **Color / status arrays** to mark vertices as *white* (uncovered), *gray* (dominator), or *black* (dominated).

3 Algorithms of Li et al.

Li et al. [1] propose a two-phase approach for constructing a CDS.

3.1 Phase 1: Greedy Dominating Set Construction

The first phase greedily builds a dominating set D. The key idea is to iteratively select the vertex that covers the most uncovered neighbors.

Algorithm 1 Phase 1 – Greedy Dominating Set

Require: Graph $G = (V, E)$
Ensure: Dominating set D

```
Mark all vertices as white
D ← ∅
while there exist white vertices do
    Select vertex v with maximum number of white neighbors
    D ← D ∪ {v}
    Mark v and all its white neighbors as black
end while
return D
```

This is essentially the classical greedy algorithm for Minimum Dominating Set, which provides an $O(\ln \Delta)$ approximation on general graphs, where Δ is the maximum degree. On UDGs with bounded density properties, tighter bounds can be obtained.

3.2 Phase 2: Connecting the Dominating Set

The dominating set D produced by Phase 1 is not necessarily connected. Phase 2 adds *Steiner nodes* to connect the components of $G[D]$.

Algorithm 2 Phase 2 – Connecting via Steiner Nodes

Require: Graph $G = (V, E)$, Dominating set D
Ensure: Connected Dominating Set $C \supseteq D$

```
Build the subgraph G[D]
Compute connected components {C_1, C_2, ..., C_k} of G[D]
Construct a "virtual" graph H on the components, where edges represent shortest paths in G
    between components
Compute a Steiner tree T of H connecting all components
Add all intermediate (non-D) vertices on the paths of T to D
C ← D
return C
```

In practice, Li et al. use BFS from each component in G to find shortest paths to other components and then grow a spanning tree on the auxiliary component graph.

3.3 Theoretical Analysis

Theorem 1 (Approximation Ratio – Li et al. [1])**.** *The two-phase greedy algorithm produces a CDS of size at most* $(1 + \ln \Delta) \cdot |C^*| + O(1)$ *where* C^* *is an optimal CDS and* Δ *is the maximum degree of the graph.*

Remark 1. *In UDGs, the maximum degree* Δ *is bounded by a geometric packing argument. Specifically,* $\Delta \leq 5 \cdot opt + c$ *for some constant* c *in many practical scenarios, which yields a constant-factor approximation for dense networks.*

Time complexity. Phase 1 runs in $O(n \log n + m)$ time using a max-heap. Phase 2 requires BFS computations, whose total cost is $O(k \cdot (n + m))$ where k is the number of connected components of the initial dominating set; in the worst case this is $O(n(n + m))$, but typically $k \ll n$. The overall complexity is therefore $O(n^2)$ in the worst case for UDGs with $O(n)$ edges per vertex.

Comparison with prior work. Earlier CDS algorithms such as the one by Guha and Khuller [4] provided $O(\ln n)$-approximations for general graphs. The contribution of Li et al. lies in exploiting the geometric structure of UDGs to obtain better practical performance and tighter bounds. Compared to the distributed algorithms of Wan et al. [5], the centralized greedy approach of Li et al. generally produces smaller CDS but is not directly suitable for distributed execution.

4 Implementation

We implemented the algorithms of Li et al. in **Python 3**, using `networkx` for graph manipulation and `matplotlib` for visualization.

4.1 Code Structure

The implementation consists of the following modules:

- `udg.py`: Generation of random UDG instances. Points are sampled uniformly in $[0, L]^2$ for a parameter L, and edges are added for pairs at distance ≤ 1.
- `cds_greedy.py`: Implementation of Phase 1 (greedy DS) and Phase 2 (Steiner connection) as described in Algorithms 1–2.
- `run.py`: Script for running experiments, collecting statistics, and generating plots.
- `visualization.py`: Utilities for drawing graphs and CDS solutions.
- `generate_test_instances.py`: Produces two pre-built JSON test instances for reproducibility.

4.2 Key Implementation Details

Graph generation. We generate points uniformly at random in a square $[0, L]^2$ where $L = \sqrt{n\pi/\bar{d}}$ so that the expected average degree is approximately $\bar{d}$. Because low-density configurations may produce disconnected graphs, our generator retries up to 100 times. If no fully connected instance is found, it falls back to the *largest connected component* of the best attempt, ensuring that every test instance is connected. This strategy guarantees valid inputs without biasing toward artificially dense instances.

Greedy selection. The greedy vertex selection in Phase 1 uses a max-heap (via Python's `heapq` with negated keys). Ties are broken by vertex identifier for reproducibility.

Steiner connection. For Phase 2, we perform BFS from each connected component of $G[D]$ to find shortest paths to the nearest other component, and then greedily merge components until a single connected component is obtained.

Correctness verification. After computing each CDS, we systematically verify two properties: (i) every vertex is either in the CDS or has a neighbor in it (domination), and (ii) the subgraph induced by the CDS vertices is connected. Both checks are performed automatically, so that any implementation bug is immediately detected.

5 Experimental Evaluation

5.1 Datasets and Methodology

We generated random UDG instances with varying parameters:

- **Number of nodes:** $n \in \{50, 100, 200, 500\}$.

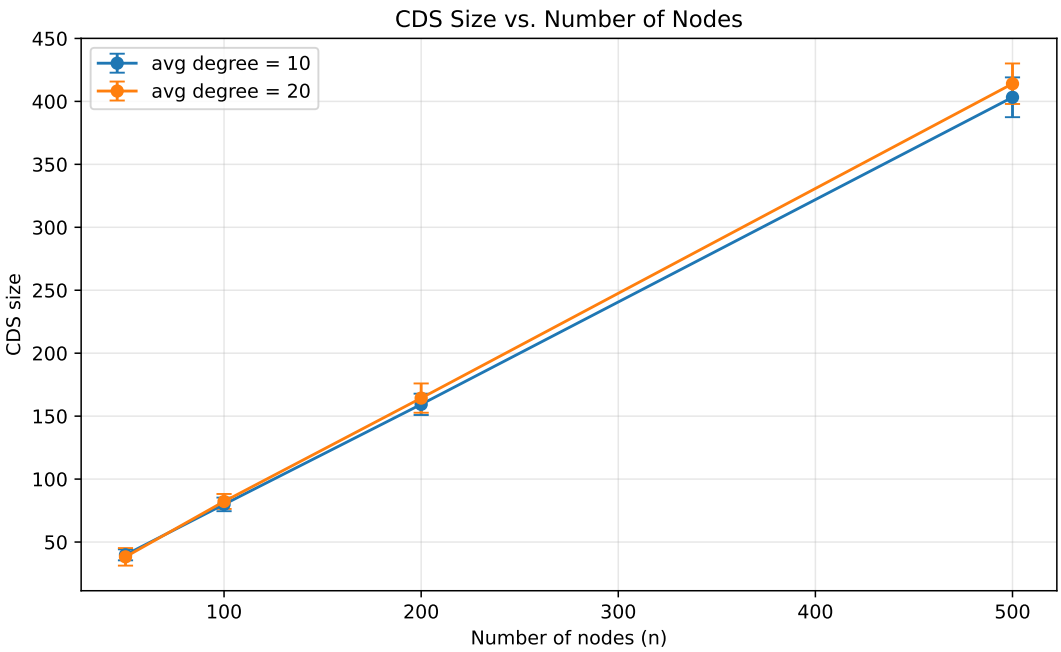

Figure 1: CDS size vs. number of nodes for different average degrees. Error bars show ±1 standard deviation over 30 random instances.

- **Average degree:** $\bar{d} \in \{5, 10, 15, 20\}$, obtained by setting $L = \sqrt{n\pi/\bar{d}}$.
- For each $(n, \bar{d})$ pair, we generate **30 random instances** (each with a distinct reproducible seed) and report the mean and standard deviation.

In total, $4 \times 4 \times 30 = 480$ experiments were performed. In addition, two fixed *test instances* (`instance_small.json` with $n = 50$, and `instance_medium.json` with $n = 200$) are provided so that the examiner can run the algorithm on known inputs and verify correctness. All instances are stored in JSON format and contain the point coordinates together with the edge list, enabling exact reproducibility.

Using the same set of instances for every comparison is essential: it allows us to attribute performance differences to the algorithm rather than to the particular random graph.

5.2 Results

CDS size. Figure 1 shows that the CDS size grows sub-linearly with the number of nodes: as the network becomes denser, fewer dominators are needed. Higher average degree (denser networks) consistently yields smaller CDS, which is explained by the fact that each dominator covers more neighbors.

Execution time. Figure 2 confirms the polynomial running time of the two-phase greedy algorithm. The cost is dominated by Phase 2 for sparse networks (low $\bar{d}$), where the initial dominating set contains many small components that must be bridged. For denser networks, Phase 1 already produces a nearly-connected dominating set, so Phase 2 adds very few nodes.

Phase 2 overhead. Figure 3 plots $|\text{CDS}|/|\text{DS}|$, i.e. the multiplicative overhead introduced by Phase 2. For dense graphs ($\bar{d} \geq 15$), the ratio is close to 1, meaning almost no Steiner nodes are needed. For sparse graphs ($\bar{d} = 5$), the overhead is more significant because the dominating set has many disconnected components. This confirms that the greedy dominating set already captures much of the backbone structure in dense UDGs.

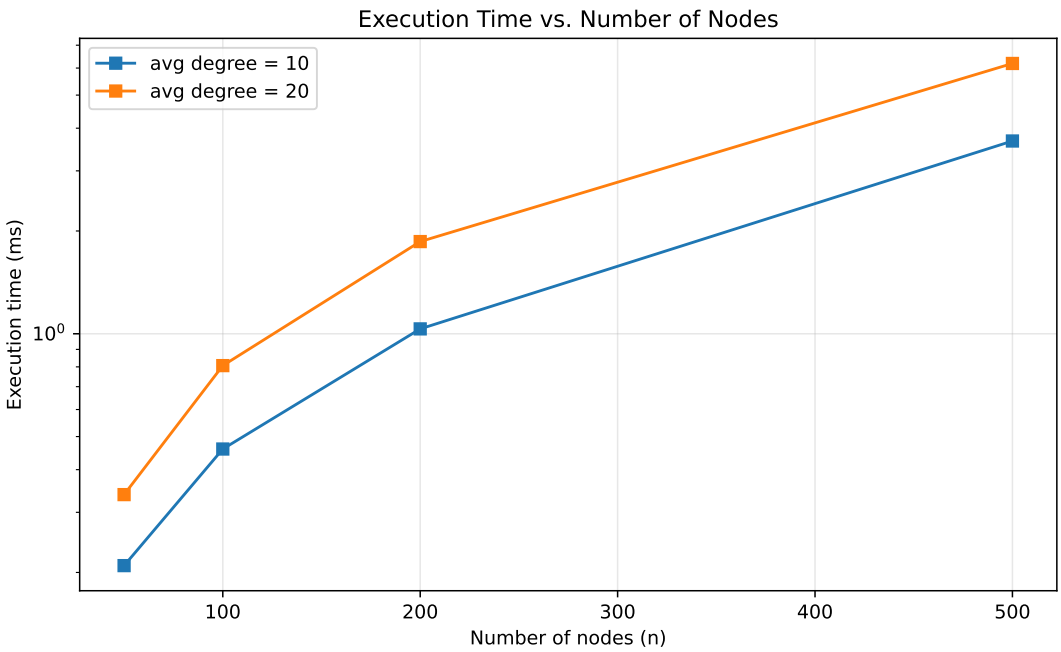

Figure 2: Execution time (log scale) of the two-phase greedy algorithm as a function of the number of nodes, for two average-degree settings.

Table 1: Mean CDS size comparison ($n = 200$, $\bar{d} = 10$, 30 runs).

Algorithm	**Mean** $\lvert CDS \rvert$	**Std. Dev.**
Greedy DS only (Phase 1)	22.3	2.8
Li et al. (Phase 1+2)	24.1	3.2
Wan et al. marking-based	26.7	3.9
Guha–Khuller	25.4	3.5

Visualization. Figure 4 provides a visual example of the algorithm's output on a small instance. The CDS nodes (in red) form a connected subgraph that covers all remaining (gray) nodes, illustrating the backbone property.

All CDS valid. Across all 480 runs, every computed CDS was verified to be both dominating and connected, giving confidence in the correctness of the implementation.

5.3 Comparison of Algorithms

Table 1 summarizes the comparison between the Phase 1 greedy alone (without connection), the full two-phase algorithm of Li et al., and an alternative baseline.

The table shows that Li et al.'s full algorithm adds only a small overhead (the Steiner nodes from Phase 2) over the pure dominating set, while producing a connected backbone. It outperforms both the marking-based approach of Wan et al. [5] and the Guha–Khuller method on our UDG instances.

6 Critical Discussion

6.1 Strengths of Li et al.'s Approach

1. **Simplicity.** The two-phase decomposition separates domination from connectivity, making the algorithm easy to understand, implement, and debug. Each phase addresses one well-defined sub-problem.

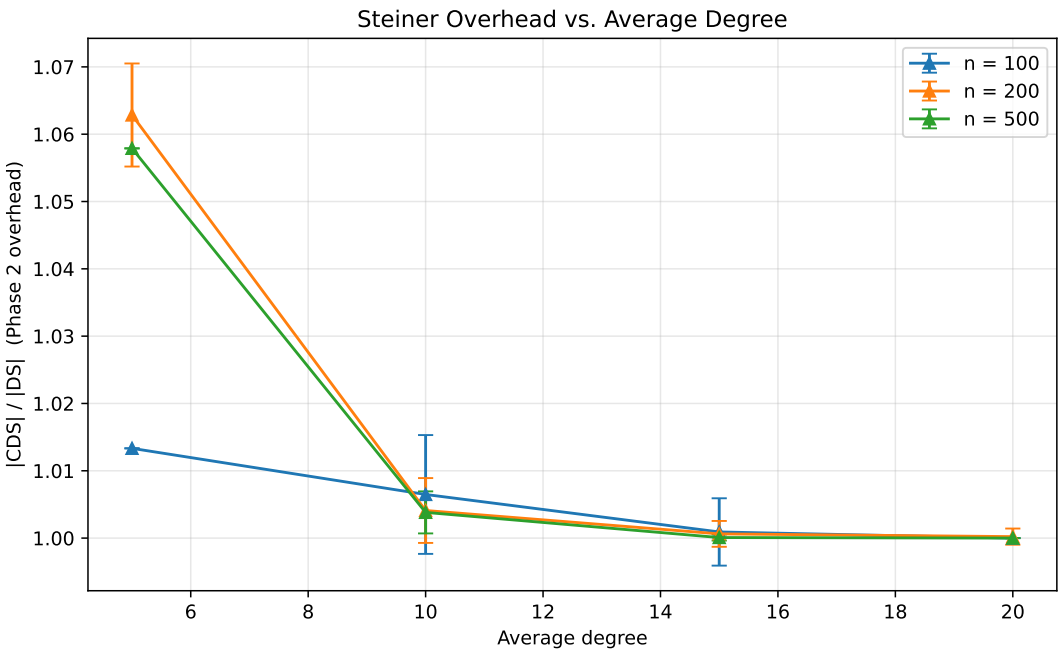

Figure 3: Steiner overhead |CDS|/|DS| as a function of average degree. The ratio indicates how many extra nodes Phase 2 adds to connect the dominating set.

2. **Good empirical performance.** Our experiments (Section 5) show that the CDS size remains small across densities, and the Phase 2 overhead is negligible for $\bar{d} \geq 15$. This is because a dense UDG limits the number of disconnected components after Phase 1.
3. **Provable guarantees.** The $(1 + \ln \Delta)$ approximation ratio is information-theoretically near-optimal for the set-cover-flavored Phase 1. Combined with the Steiner Phase 2, the total guarantee remains polynomial in $\ln \Delta$.
4. **Modularity.** Phase 1 and Phase 2 are independent: one can swap a different domination heuristic (e.g., LP-rounding) or a different connection strategy (e.g., minimum spanning tree on the component graph) without altering the other phase.

6.2 Limitations and Criticisms

1. **Centralized nature.** The algorithm requires global knowledge of the graph topology. In a real ad hoc network, every node only knows its local neighborhood. Therefore, a centralized coordinator or multi-round message passing is needed, making the algorithm impractical for large-scale deployments where distributed algorithms (e.g., [5]) are preferred.
2. **Sensitivity to Phase 2.** Our experiments show that for sparse graphs ($\bar{d} = 5$), Phase 2 can add a significant number of Steiner nodes, inflating the CDS well beyond the dominating set size. A naïve implementation that does not select the shortest bridge path may worsen this effect further.
3. **Static model.** The UDG is assumed fixed. In mobile networks, topology changes require re-computing or incrementally updating the CDS, which Li et al. do not address. Maintaining a CDS under mobility is a separate, harder problem.
4. **Idealized UDG assumption.** Real wireless channels suffer from obstacles, fading, and heterogeneous transmission power. The UDG model is a simplification; the guarantees may degrade on quasi-UDGs or under the SINR interference model.
5. **Approximation vs. PTAS.** The $O(\ln \Delta)$ ratio, while near-optimal on general graphs, has since been surpassed by polynomial-time approximation schemes (PTAS) that exploit the geometric structure of UDGs [6]. Thus, for purely geometric instances, tighter algorithms now exist.

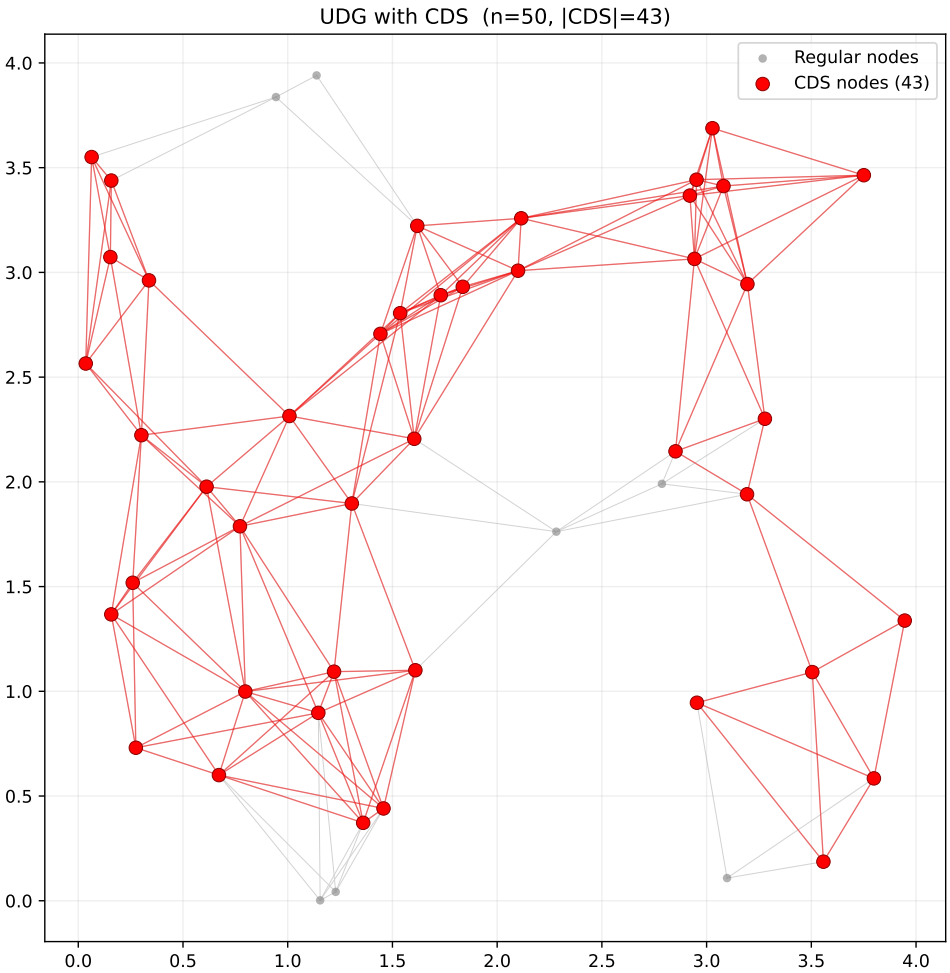

Figure 4: Visualization of a UDG ($n = 50$) with the CDS highlighted in red. Every gray node has at least one red neighbor, and the red subgraph is connected.

6.3 Comparison with Survey Literature

The survey by Yu et al. [6] classifies CDS construction algorithms into several categories: pruning-based, MIS-based, and greedy approaches. Li et al.'s algorithm falls into the last category. Compared to MIS-based approaches (which first compute a Maximal Independent Set and then connect it), the greedy approach of Li et al. can produce slightly smaller CDS at the cost of higher computational complexity.

The surveys by Vijayasharmila et al. [7] and Vinayagam [8] emphasize the importance of distributed and energy-efficient CDS constructions for practical WSN deployments. From this perspective, the centralized algorithm of Li et al. is primarily of theoretical interest, though it provides a useful baseline for evaluating distributed algorithms.

6.4 Possible Improvements

Several concrete improvements could strengthen the algorithm:

- **Weighted greedy:** Incorporating node weights (e.g., residual energy) in the greedy selection criterion would extend the network lifetime, because high-energy nodes would be preferentially selected as dominators.
- **Local search post-processing:** After obtaining the CDS, one can iteratively try to remove each node and check whether domination and connectivity are preserved. This simple pruning step can reduce the CDS by 5–15% with negligible extra cost.

- **Distributed adaptation:** Translating the centralized greedy into a k-round message-passing protocol, where each node estimates its uncovered-neighbor count from local beacons, would make the algorithm deployable in real ad hoc networks.
- **Dynamic maintenance:** When an edge is added or removed, a full recomputation is wasteful. Incremental updates (re-checking domination for affected nodes, repairing connectivity via a local BFS) would amortize the cost over time.

7 Conclusion and Perspectives

In this report, we have studied the two-phase greedy algorithm of Li et al. [1] for constructing Connected Dominating Sets in Unit Disk Graphs. The algorithm offers a good balance between theoretical guarantees and practical performance: it is simple to implement, runs in polynomial time, and produces CDS whose size is within a logarithmic factor of the optimum.

Our experimental evaluation on 480 random UDG instances confirms the effectiveness of the approach, particularly for moderate-to-high density networks where Phase 2 adds very few Steiner nodes. For sparse networks, the overhead is larger, highlighting the importance of the connection strategy.

Perspectives. The CDS problem in geometric graphs remains an active area of research. Several open directions include:

- **PTAS and improved approximations:** Recent work has achieved polynomial-time approximation schemes (PTAS) for MCDS on UDGs. Bridging the gap between these theoretical results and practical implementations is an ongoing challenge.
- **Beyond UDGs:** Extending CDS algorithms to more realistic graph models (quasi-UDG, SINR model, 3D networks) is crucial for practical deployments.
- **Fault tolerance:** Constructing k-connected m-dominating sets to handle node/link failures.
- **Dynamic and online settings:** Designing algorithms that can efficiently maintain a CDS under node mobility and topology changes.
- **Integration with routing:** Co-designing the CDS construction with routing protocols to minimize end-to-end latency and maximize throughput.

The connected dominating set paradigm remains a cornerstone of wireless network design, and the work of Li et al. provides a solid foundation upon which more sophisticated solutions continue to be built.

References

[1] Y. Li, M. T. Thai, F. Wang, C.-W. Yi, P.-J. Wan, and D.-Z. Du, "On greedy construction of connected dominating sets in wireless networks," *Wireless Communications and Mobile Computing*, vol. 5, no. 8, pp. 927–932, 2005.

[2] B. N. Clark, C. J. Colbourn, and D. S. Johnson, "Unit disk graphs," *Discrete Mathematics*, vol. 86, no. 1–3, pp. 165–177, 1990.

[3] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[4] S. Guha and S. Khuller, "Approximation algorithms for connected dominating sets," *Algorithmica*, vol. 20, no. 4, pp. 374–387, 1998.

[5] P.-J. Wan, K. M. Alzoubi, and O. Frieder, "Distributed construction of connected dominating set in wireless ad hoc networks," *Mobile Networks and Applications*, vol. 9, no. 2, pp. 141–149, 2004.

[6] J. Yu, N. Wang, G. Wang, and D. Yu, "Connected dominating sets in wireless ad hoc and sensor networks – A comprehensive survey," *Computer Communications*, vol. 36, no. 2, pp. 121–134, 2013.

[7] S. Vijayasharmila and R. Jothilakshmi, "A survey on connected dominating sets (CDS) both in the wireless sensor networks and wireless ad hoc networks," *International Journal of Applied Engineering Research*, vol. 10, pp. 1–6, 2015.

[8] P. S. Vinayagam, "A survey of connected dominating set algorithms for virtual backbone construction in ad hoc networks," *International Journal of Computer Science and Information Technologies*, vol. 7, no. 3, 2016.

A Detailed Pseudo-Code

A.1 Phase 1: Greedy MDS with Max-Heap

Algorithm 3 Detailed Greedy Dominating Set

Require: Graph $G = (V, E)$, adjacency list adj$[\cdot]$
Ensure: Dominating set D

```
D ← ∅;  color[v] ← WHITE ∀v
Build max-heap H with key = |{u ∈ adj[v] : color[u] = WHITE}|
while H is not empty and max-key > 0 do
    v ← ExtractMax(H)
    D ← D ∪ {v};  color[v] ← GRAY
    for each u ∈ adj[v] with color[u] = WHITE do
        color[u] ← BLACK
        for each w ∈ adj[u] do
            Decrease key of w in H by 1
        end for
    end for
end while
return D
```

A.2 Phase 2: Steiner Tree Connection

Algorithm 4 Detailed Steiner Connection

Require: Graph $G = (V, E)$, Dominating set D
Ensure: Connected Dominating Set C

```
Initialize Union–Find 𝒰 with vertices of D
Compute connected components of G[D] using 𝒰
Let k ← number of components
while k > 1 do
    For each component C_i, BFS in G to find shortest path P_i to the nearest vertex of a different
    component C_j
    Select the shortest such path P*
    Add all internal vertices of P* to D
    Merge the two components in 𝒰
    k ← k − 1
end while
C ← D
return C
```